ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES
(AUTONOMOUS)
(Permanent Affiliation by Andhra University & Approved by AICTE
Accredited by NBA (ECE, EEE, CSE, IT, Mech, Civil & Chemical) & NAAC with A+ Grade)
Sangivalasa-531162, Bheemunipatnam Mandal,
Visakhapatnam District

A PROJECT WORK REPORT ON..

E-VOTING SYSTEM

This project is submitted by

A.JAHNAVI	A25126510001
B.DIVYA	A25126510003
N.RAGA PRIYAMBICA	A25126510031
V.BABY LAYA	A25126510063

Under the Guidance of
Dr B.MAHESH
Department of Computer Science and Engineering

in partial fulfillment of the requirements for the award of the
offered by

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

PRAGNANAM BRAHMA
ANITS

Bachelor of Technology
in
COMPUTER SCIENCE AND ENGINEERING
2024-2028

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES
(AUTONOMOUS)
(Permanent Affiliation by Andhra University & Approved by AICTE
Accredited by NBA (ECE, EEE, CSE, IT, Mech, Civil & Chemical) & NAAC with A+ Grade)
Sangivalasa-531162, Bheemunipatnam Mandal,
Visakhapatnam District

CERTIFICATE

This is to certify that this project work entitled — "**E-VOTING SYSTEM**" is the bonafied record work done by **Ms.A.Jahnavi,Ms.B.Divya,Ms.N.Raga priyambica ,Ms.V.Baby laya** bearing Roll no: **A25126510001,A25126510003,A25126510031,A25126510063** of the first year , along with batch mates submitted on the partial fulfillment of the requirement for the award of B.tech in Computer Science and Engineering to the ANIL NEEKURONDA INSTITUTE OF TECHNOLOGY AND SCIECNCES. The results embodied in this project report have not been submitted to any other board/University or Institute for the award of diploma.

HEAD OF THE DEPARTMENT

LECTURER IN CHARGE

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this project.

First and foremost, I extend my heartfelt thanks to my project guide,
DR B.MAHESH, for their valuable guidance, continuous support, and encouragement throughout the development of this project. Their insightful feedback and expert advice were instrumental in shaping this work.
I am also grateful to the faculty members of Computer Science and Engineering, ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES, for providing the necessary resources and a conducive environment for research and learning.

A special thanks to my friends and classmates for their cooperation, constructive suggestions, and moral support during every stage of this project.

Finally, I would like to thank my family for their unwavering love, patience, and constant motivation that kept me focused and determined.

This project, titled “E-VOTING SYSTEM”, has been a great learning experience, helping me enhance my technical skills and practical understanding of the subject.

ABSTRACT

The Electronic Voting System (E-Voting System) is an advanced digital solution designed to modernize the traditional voting process by replacing paper-based voting with electronic methods. The main purpose of this system is to provide a secure, efficient, transparent, and reliable way of conducting elections. In the traditional voting system, the process of registration, vote casting, and counting takes a lot of time and resources, and there are chances of human errors and malpractice. The E-Voting System helps overcome these problems by automating the entire voting process.

This system allows voters to register and verify their identity using unique credentials such as voter ID, password, or biometric authentication. After successful verification, the voter can cast their vote through an electronic platform such as a computer or mobile device. The system ensures that each voter can vote only once and that the vote remains confidential. Security measures like encryption and authentication protect the voting data from unauthorized access or tampering.

One of the major advantages of the E-Voting System is faster vote counting and instant result generation, which reduces delays in announcing election results. It also minimizes paperwork, saves time, reduces operational costs, and makes the election process more environmentally friendly. Additionally, it improves accessibility by allowing people in remote areas or with physical disabilities to participate more easily in elections.

The E-Voting System increases transparency and builds trust among voters by ensuring fairness and accuracy in the election process. Overall, it is an effective and innovative solution for conducting secure and efficient elections in the digital era.

PREFACE

Table of contents

Sl.No.	Chapter Title	Sub-Contents	Page No.
1	Introduction	• Overview of the Project • Purpose and Scope • Existing System and Proposed System	1
2	Problem Definition	• Problem Statement • Objectives of the System	2
3	System Design	• System Architecture • Flowcharts • Algorithms Used	3-9
4	Implementation Details	• Program Structure • Language and Environment Used • Key Functional Modules • Code Snippets and Screenshots • File Handling Explanation	10-13
5	Results and Discussion	• Output Screens • Test Case Table (Input, Expected Output, Actual Output)	14-17
6	Conclusion	• Summary of Achievements • Limitations of the System • Future Enhancements	18-19

7	References	• Books Referred • Research Papers • Websites and Online Resources	20
8	Appendix	• Source Code Listings • Source Code	

1.INTRODUCTION

Overview

The E-Voting System is a software application developed in C++ to conduct elections electronically. It replaces the traditional paper ballot system with a digital process, making voting faster, easier, and more reliable. In this system, voters can cast their votes by selecting their preferred candidate through the application.

The system stores voter details, candidate details, and vote counts securely using file handling concepts in C++. It ensures that each voter can vote only once, thereby maintaining fairness in the election process.

Electronic voting systems are useful in schools, colleges, organizations, and small institutions where elections are conducted frequently.

Purpose and Scope

The main purpose of this project is to create a secure and efficient voting system using C++ programming.

The scope of the project includes:

Conducting elections digitally

Reducing paper usage

Minimizing human errors

Generating results quickly

Maintaining voter records

This system can be used for student elections, office elections, and organizational voting.

Existing System

In the traditional voting system:

Voting is done using paper ballots.

Counting votes takes more time.

There is a chance of invalid votes.

More manpower is required.

Maintaining records is difficult.

Proposed System

The proposed E-Voting System solves these problems by:

Providing digital voting.

Reducing manual effort.

Counting votes automatically.

Preventing duplicate voting.

Storing records digitally.

2.Problem Definition

Problem Definition – E-Voting System

The traditional voting system is mainly paper-based, where voters cast their votes manually and the votes are counted by election officials. This process takes a lot of time, requires more manpower, and may lead to human errors during vote counting. There is also a possibility of invalid votes, duplicate voting, and difficulties in maintaining records securely.

In many organizations, colleges, and institutions, conducting elections using the manual system becomes difficult because it is time-consuming and less efficient. Managing voter information and ensuring transparency in the voting process is also a major challenge.

To overcome these issues, an Electronic Voting System is proposed. The E-Voting System is designed to automate the voting process using C++ programming. It allows voters to cast their votes electronically, stores the data securely, and calculates results automatically. The system ensures that each voter can vote only once, improves accuracy, saves time, and reduces paperwork.

The main goal of the E-Voting System is to make the election process simple, secure, transparent, and efficient.

3.System Design

The system design of the E-Voting System defines how the application is structured and how different modules work together to conduct the voting process efficiently.

1. System Architecture

The E-Voting System is divided into the following modules:

Admin Module:
This module allows the administrator to manage the election process. The admin can add candidates, view voter details, and check results.

Voter Module:
This module is used by voters to register and cast their votes. Each voter is verified before voting.

Candidate Module:
This module stores candidate information such as candidate ID, name, and vote count.

Voting Module:
This module allows voters to select a candidate and cast their vote securely.

Result Module:
This module counts the votes and displays the final election result.

---

2. Flow of the System

The working process of the system is as follows:

1. Start the system
2. Display main menu
3. Register voter
4. Verify voter details
5. Display candidate list
6. Cast vote
7. Store vote data
8. Count votes
9. Display results
10. End system

3. Input Design

The input design includes:
Voter ID
Voter Name
Candidate Selection
Login Credentials
The system accepts user input and validates it.

---

4. Output Design

The output includes:
Confirmation message after voting
Candidate vote count
Winner details
Final election result

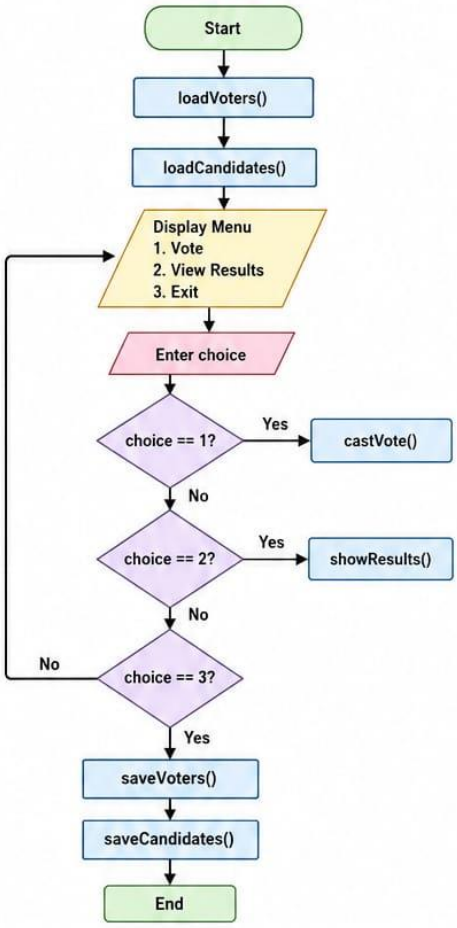
---

5. Data Storage Design

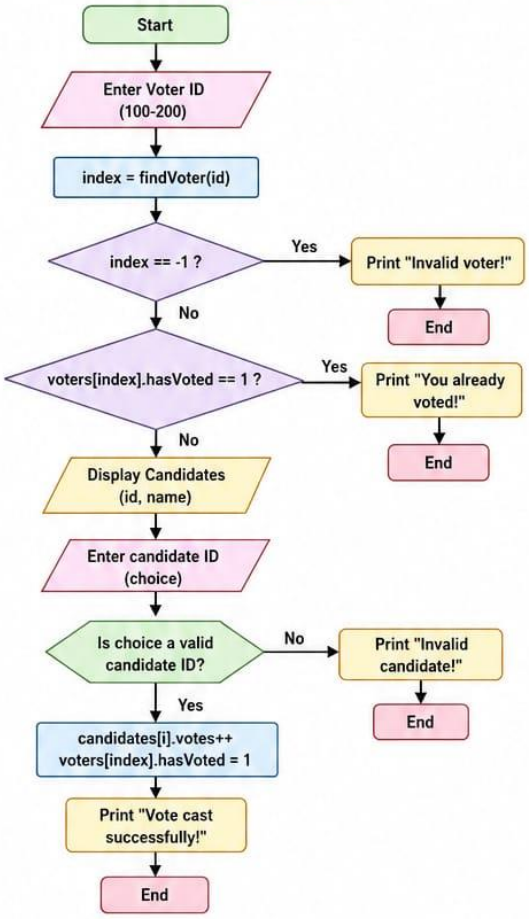
The system uses file handling in C++ to store:p
Voter details
Candidate details
Vote records
Files help maintain data permanently for future reference.

E-VOTING SYSTEM
(FLOWCHARTS)

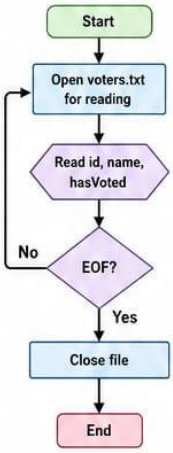
MAIN FLOWCHART



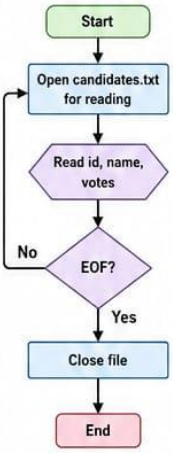
CAST VOTE FLOWCHART



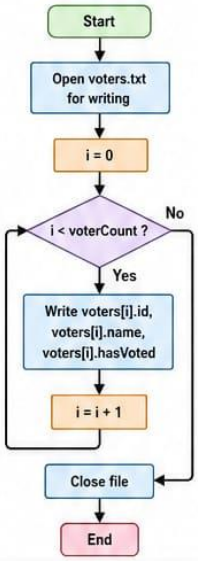
LOAD VOTERS



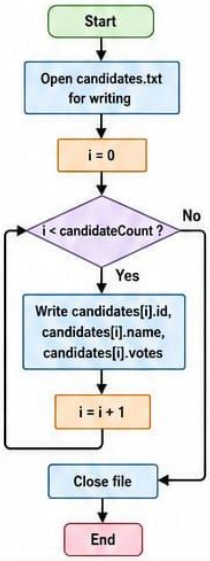
LOAD CANDIDATES



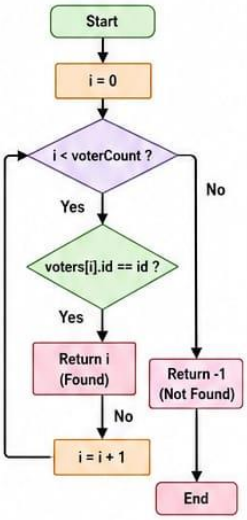
SAVE VOTERS



SAVE CANDIDATES



FIND VOTER (id)



4.IMPLEMENTATION DETAILS

Step 1: Include Header Files

```
#include <iostream>
#include <fstream>
using namespace std;
```

This part of the code is used to include the required libraries for the program. The iostream header file is used for taking input from the user and displaying output on the screen. The fstream header file is used for file handling operations such as reading data from files and writing data into files. The statement using namespace std; is used so that standard functions like cout, cin, and ifstream can be used directly without writing std:: before them.

Step 2: Create Voter Class

```
class Voter {
public:
    int id;
    string name;
    int hasVoted;
};
```

This class is created to store the information of voters. It contains three data members: id, name, and hasVoted. The id variable stores the voter identification number, name stores the voter name, and hasVoted is used to check whether the voter has already voted or not. If the value is 0, it means the voter has not voted yet, and if the value is 1, it means the voter has already voted.

Step 3: Create Candidate Class

```
class Candidate {
public:
    int id;
    string name;
    int votes;
};
```

This class is used to store the details of candidates participating in the election. It contains candidate ID, candidate name, and the number of votes received. The votes variable increases whenever a voter selects that candidate.

Step 4: Declare Arrays and Variables

```
Voter voters[100];
Candidate candidates[10];
int voterCount = 0;
int candidateCount = 0;
```

Arrays are declared to store multiple voter and candidate records. The voters array can store up to 100 voters, and the candidates array can store up to 10 candidates. The variables voterCount and candidateCount are used to keep track of how many records are loaded from the files.

Step 5: Load Voter Data

```
void loadVoters() {
    ifstream file("voters.txt");
    while (file >> voters[voterCount].id >> voters[voterCount].name >> voters[voterCount].hasVoted) {
        voterCount++;
    }
    file.close();
}
```

This function is used to load voter data from the file named voters.txt. It reads voter ID, name, and voting status from the file and stores them into the voter array. This helps the program to access previously saved voter information whenever the program starts.

Step 6: Load Candidate Data

```
void loadCandidates() {
    ifstream file("candidates.txt");
    while (file >> candidates[candidateCount].id >> candidates[candidateCount].name >> candidates[candidateCount].votes) {
        candidateCount++;
    }
    file.close();
}
```

This function reads candidate information from the file candidates.txt. It loads candidate ID, candidate name, and current vote count into the candidate array. This allows the system to continue from previously stored data.

Step 7: Save Voter Data

```
void saveVoters() {
    ofstream file("voters.txt");
    for (int i = 0; i < voterCount; i++) {
        file << voters[i].id << " " << voters[i].name << " " << voters[i].hasVoted << endl;
    }
    file.close();
}
```

This function is used to store updated voter details into the file voters.txt. When a voter casts a vote, their voting status changes, and this updated information is saved in the file so that it remains permanent.

Step 8: Save Candidate Data

```
void saveCandidates() {
    ofstream file("candidates.txt");
    for (int i = 0; i < candidateCount; i++) {
        file << candidates[i].id << " " << candidates[i].name << " " << candidates[i].votes << endl;
    }
    file.close();
}
```

This function saves the latest vote counts of all candidates into the file candidates.txt. Whenever a vote is cast, the candidate's vote count increases, and this updated result is stored in the file before exiting the program.

Step 9: Find Voter

```
int findVoter(int id) {
    for (int i = 0; i < voterCount; i++) {
        if (voters[i].id == id) {
            return i;
        }
    }
    return -1;
}
```

This function is used to search for a voter using the entered voter ID. It checks all records in the voter array. If the voter ID is found, it returns the index position of that voter. If the voter ID is not found, it returns -1, which means the voter is invalid.

Step 10: Cast Vote Function

```
void castVote() {
    int id;
    cout << "Enter Voter ID(100-200): ";
    cin >> id;
    int index = findVoter(id);
    if (index == -1) {
        cout << "Invalid voter!\n";
        return;
    }
    if (voters[index].hasVoted == 1) {
        cout << "You already voted!\n";
        return;
    }
    cout << "\nCandidates:\n";
    for (int i = 0; i < candidateCount; i++) {
        cout << candidates[i].id << ". " << candidates[i].name << endl;
    }
    int choice;
    cout << "Enter candidate ID: ";
    cin >> choice;
    for (int i = 0; i < candidateCount; i++) {
        if (candidates[i].id == choice) {
            candidates[i].votes++;
            voters[index].hasVoted = 1;
            cout << "Vote cast successfully!\n";
            return;
        }
    }
    cout << "Invalid candidate!\n";
}
```

This is the most important function in the program because it handles the voting process. First, it

asks the user to enter the voter ID. Then it checks whether the voter exists or not. If the voter is invalid, it shows an error message. If the voter has already voted, it informs that voting is not allowed again. If the voter is valid and has not voted, the program displays the list of candidates. The voter selects a candidate by entering candidate ID. Then the selected candidate's vote count increases by one, and the voter status changes to voted.

Step 11: Show Results

```
void showResults() {
    cout << "\n--- Voting Results ---\n";
    for (int i = 0; i < candidateCount; i++) {
        cout << candidates[i].name << " : " << candidates[i].votes << " votes\n";
    }
}
```

This function is used to display the election results. It shows each candidate's name along with the number of votes they have received. This helps users to view the current voting status.

Step 12: Main Function
Save Data Before Exit

```
int main() {
    loadVoters();
    loadCandidates();
    int choice;
    do {
        cout << "\n--- E-Voting System ---\n";
        cout << "1. Vote\n";
        cout << "2. View Results\n";
        cout << "3. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;
        if (choice == 1) {
            castVote();
        }
        else if (choice == 2) {
            showResults();
        }
    } while (choice != 3);
    saveVoters();
    saveCandidates();
    return 0;
}
```

The main() function is the starting point of the program. First, it calls loadVoters() and loadCandidates() to read stored data. Then it repeatedly displays a menu with three options: Vote, View Results, and Exit. If the user chooses Vote, it calls the castVote() function. If the user chooses View Results, it calls showResults(). The loop continues until the user selects Exit. Before closing the program, these functions are called to save all updated voter records and candidate vote counts into files. This ensures that no data is lost after the program ends.

5.OUTPUT SCREENS

1.Main menu screen

```
--- E-Voting System ---
1. Vote
2. View Results
3. Exit
Enter choice: 1
Enter Voter ID(100-200): 101
You already voted!

--- E-Voting System ---
1. Vote
2. View Results
3. Exit
Enter choice: 1
Enter Voter ID(100-200): 102
You already voted!
```

2.Casting a vote

```
--- E-Voting System ---
1. Vote
2. View Results
3. Exit
Enter choice: 1
Enter Voter ID(100-200): 179

Candidates:
1. Jagan
2. Babu
3. Pavan
4. Modi
Enter candidate ID: 3
Vote cast successfully!

--- E-Voting System ---
1. Vote
2. View Results
3. Exit
```

3.Display voting results

```
Enter choice: 2

--- Voting Results ---
Jagan : 3 votes
Babu : 1 votes
Pavan : 2 votes
Modi : 1 votes

--- E-Voting System ---
1. Vote
2. View Results
3. Exit
Enter choice: 3
```

4.Second vote submission

```
--- E-Voting System ---
1. Vote
2. View Results
3. Exit
Enter choice: 2

--- Voting Results ---
Jagan : 3 votes
Babu : 1 votes
Pavan : 1 votes
Modi : 1 votes
```

5.Final voting results and exit

```
Enter choice: 2

--- Voting Results ---
Jagan : 3 votes
Babu : 1 votes
Pavan : 2 votes
Modi : 1 votes

--- E-Voting System ---
1. Vote
2. View Results
3. Exit
Enter choice: 3
```

6.CONCLUSION

The development of this “C++ E-Voting System”serves as a successful proof-of-concept for digitizing and securing the electoral process. By leveraging the efficiency of C++ and the structured approach of Object-Oriented Programming, the system addresses the primary challenges of traditional voting: accessibility, speed, and manual counting errors.

“Key Takeaways”

Security & Integrity:”Through the implementation of unique Voter ID verification and status flags, the system effectively eliminated the possibility of “double-voting,” ensuring the “one person, one vote” principle is upheld.

Efficiency:The transition from physical paper ballots to a digital interface significantly reduces the time required for both casting votes and generating final results. The automated tallying system ensures 100% accuracy in the count, provided the input data is valid.

Data Persistence: The use of C++ file handling (fstream) ensures that all election data is stored permanently in a non-volatile format, allowing for audits and recovery in the event of system interruptions.

Final Verdict

While this version of the system operates in a console-based environment, it provides a solid foundation for more complex systems. It demonstrates that even with a low-resource footprint, a program can manage high-stakes data securely.

Future Scope

To move beyond a prototype stage, future iterations could focus on:

1. Graphical User Interface (GUI):Implementing libraries like Qt or Graphics.h to make the system more user-friendly for the general public.
2. Network Integration: Transitioning from a standalone local application to a client-server architecture using Socket Programming to allow voting from multiple terminals.
3. Advanced Encryption: Incorporating SHA-256 hashing or RSA encryption to protect the voter database from advanced external tampering.
4. Biometric Authentication: Integrating hardware modules for fingerprint or facial recognition to further enhance voter identity verification.

In conclusion, this project effectively demonstrates how computational logic can be applied to strengthen democratic transparency and operational efficiency.

REFERENCES

1.Data Structures Using C++ – For understanding arrays, file handling, and program structure used in the system.
2.Object-Oriented Programming with C++ by E. Balagurusamy – Used for implementing classes like Voter and Candidate.
3.Visual Studio Code – Development environment used for writing and testing the C++ code.
4.GNU Compiler Collection – Used to compile and execute the program.
5.Election Commission of India – Reference for real-world voting procedures and rules.
Online resources:
•Geeksforgeeks – For C++ concepts and file handling.
•W3Schools – Basic programming references

1. APPENDIX

Source code:

```
#include <iostream>
#include <fstream>
Using namespace std;
Class Voter {
Public:
   Int id;
   String name;
   Int hasVoted;
};
Class Candidate {
Public:
   Int id;
   String name;
   Int votes;
};
Voter voters[100];
Candidate candidates[10];
Int voterCount = 0;
Int candidateCount = 0;
Void loadVoters() {
   Ifstream file("voters.txt");
   While (file >> voters[voterCount].id >> voters[voterCount].name >>
voters[voterCount].hasVoted) {
      voterCount++;
   }
   File.close();
}
Void loadCandidates() {
   Ifstream file("candidates.txt");
   While (file >> candidates[candidateCount].id >> candidates[candidateCount].name >>
candidates[candidateCount].votes) {
      candidateCount++;
```

```
    }
    File.close();
}
Void saveVoters() {
    Ofstream file("voters.txt");
    For (int I = 0; I < voterCount; i++) {
        File << voters[i].id << " " << voters[i].name << " " << voters[i].hasVoted << endl;
    }
    File.close();
}
Void saveCandidates() {
    Ofstream file("candidates.txt");
    For (int I = 0; I < candidateCount; i++) {
        File << candidates[i].id << " " << candidates[i].name << " " << candidates[i].votes << endl;
    }
    File.close();
}
Int findVoter(int id) {
    For (int I = 0; I < voterCount; i++) {
        If (voters[i].id == id) {
            Return I;
        }
    }
    Return -1;
}
Void castVote() {
    Int id;
    Cout << "Enter Voter ID(100-200): ";
    Cin >> id;
    Int index = findVoter(id);
    If (index == -1) {
        Cout << "Invalid voter!\n";
        Return;
    }
    If (voters[index].hasVoted == 1) {
```

```
        Cout << "You already voted!\n";
        Return;
    }
    Cout << "\nCandidates:\n";
    For (int I = 0; I < candidateCount; i++) {
        Cout << candidates[i].id << ". " << candidates[i].name << endl;
    }
    Int choice;
    Cout << "Enter candidate ID: ";
    Cin >> choice;
    For (int I = 0; I < candidateCount; i++) {
        If (candidates[i].id == choice) {
            Candidates[i].votes++;
            Voters[index].hasVoted = 1;
            Cout << "Vote cast successfully!\n";
            Return;
        }
    }
    Cout << "Invalid candidate!\n";
}
Void showResults() {
    Cout << "\n--- Voting Results ---\n";
    For (int I = 0; I < candidateCount; i++) {
        Cout << candidates[i].name << " : " << candidates[i].votes << " votes\n";
    }
}
Int main() {
    loadVoters();
    loadCandidates();
    int choice;
    do {
        cout << "\n--- E-Voting System ---\n";
        cout << "1. Vote\n";
        cout << "2. View Results\n";
        cout << "3. Exit\n";
```

```
        cout << “Enter choice: “;
        cin >> choice;
        if (choice == 1) {
            castVote();
        }
        Else if (choice == 2) {
            showResults();
        }
    } while (choice != 3);
    saveVoters();
    saveCandidates();
    return 0;
}
```